Állapotgép megvalósítás

Benzinkút

Gregorics Tibor

gt@inf.elte.hu

http://people.inf.elte.hu/gt/oep

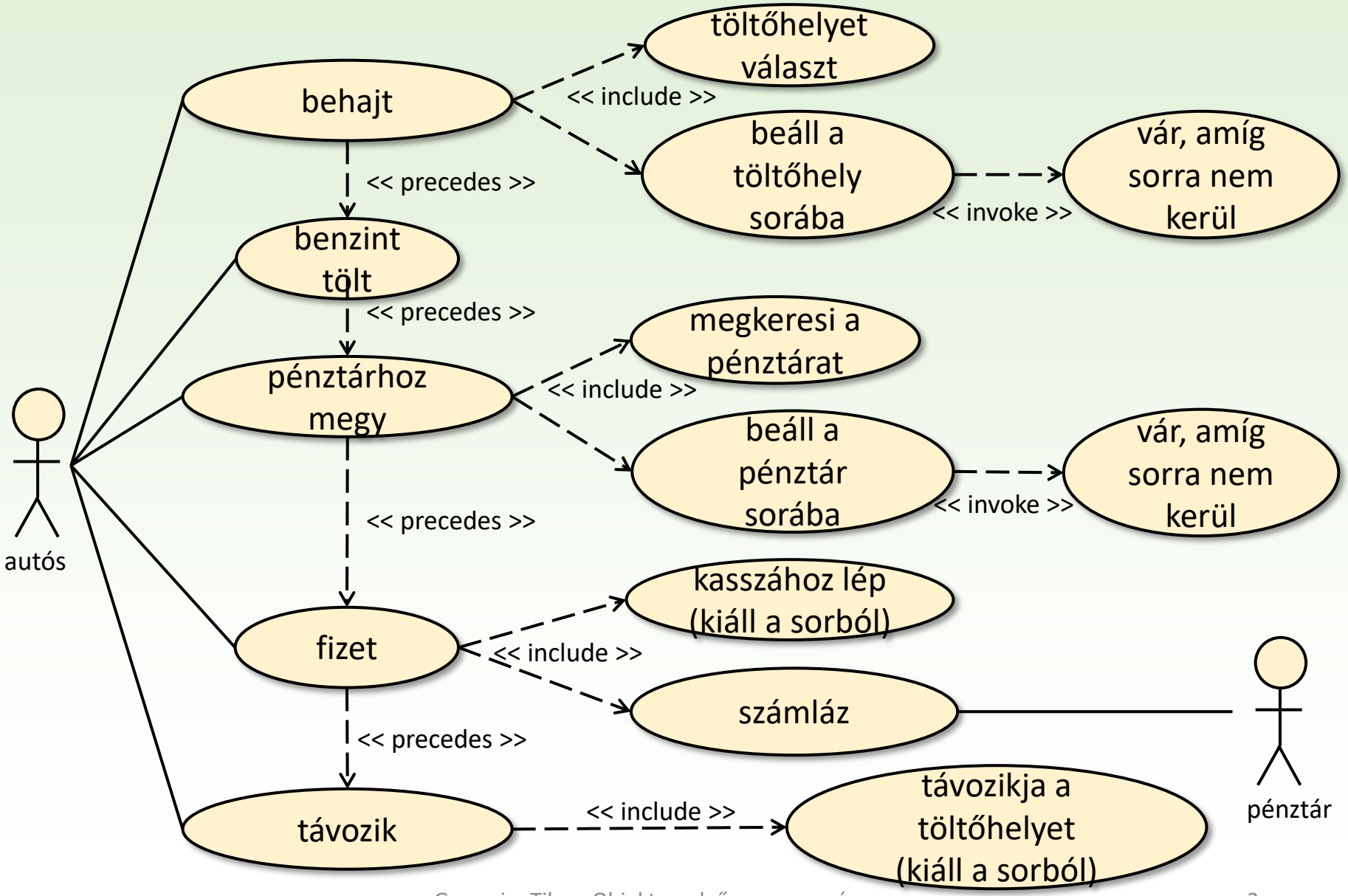
Feladat

- Egy benzinkútnál több töltőhely és egy több kasszából álló pénztár működik.
  - Az autósok behajtanak, és beállnak valamelyik töltőhelyhez tankolni.
  - Amikor sorra kerülnek, akkor kívánt mennyiségű benzint vesznek fel.
  - Ezután elmennek fizetni: beállnak a pénztárhoz várakozók sorába.
  - Amint egy kassza szabad lesz, kiszámolják soron következő autós által fizetendő összeget.
  - Fizetés után az autós kihajt a töltőhelyről, és távozik.



- Szimuláljuk ezt a folyamatot tetszőleges számú, egymással párhozamosan tevékenykedő autós esetére.

Használati eset diagram



Felhasználói esetek

eset		leírás
behajt	GIVEN	létezik a benzinkút töltőhelyekkel
	WHEN	behajt egy létező töltőhelyhez
	THEN	besorol a töltőhely melletti sorba
benzint tölt	GIVEN	az egyik töltőhelyen elsőként áll
	WHEN	megadott liter benzin töltése
	THEN	a kijelző mutatja a felvett benzint
pénztárhoz megy	GIVEN	létezik a benzinkút pénztárral
	WHEN	bemegy a pénztárba
	THEN	beáll a pénztár sorába
fizetés	GIVEN	létezik a benzinkút pénztárral, egy töltőhelyen áll, a pénztárban sorban áll
	WHEN	sorra kerül (első a sorban és valamelyik kassza üres)
	THEN	kiáll a sorból, kiszámolják a fizetendő összeget, lenullázzák a töltőhely kijelzőjét
távozik	GIVEN	a benzinkút egyik töltőhelyén áll
	WHEN	távozik
	THEN	kiáll a töltőhely sorából

eset		leírás
benzint tölt	GIVEN	az egyik töltőhelyen elsőként áll
	WHEN	nulla liter benzin töltése
	THEN	figyelmeztetés
távozik	GIVEN	töltőhelyen áll, a kijelző nem nulla
	WHEN	távozik
	THEN	riasztás

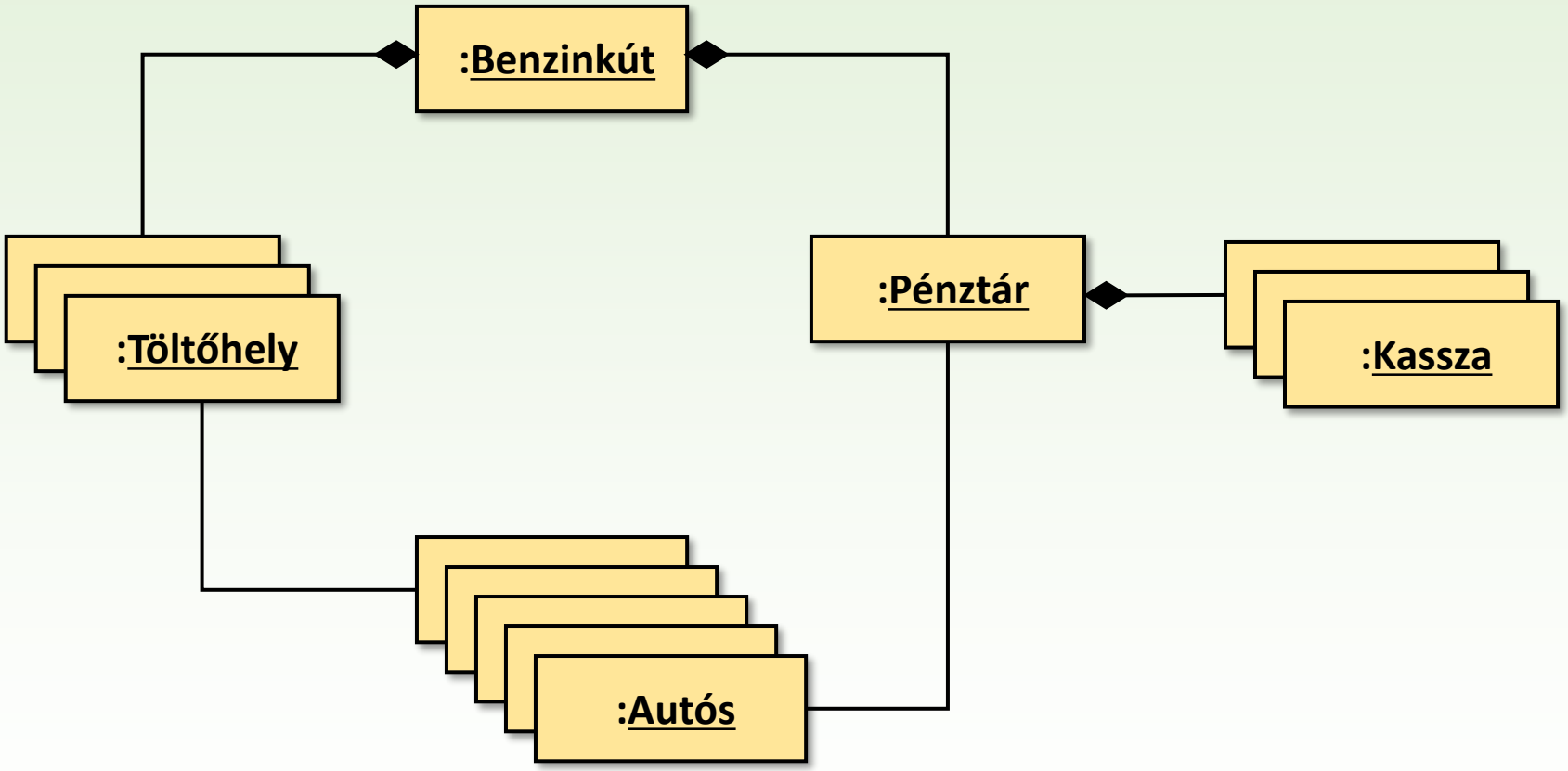
Felhasználói (hibás) esetek

eset		leírás
behajt	GIVEN	nem létezik a benzinkút vagy nem létezik a kiválasztott töltőhely
	WHEN	behajt
	THEN	hibajelzés
benzint tölt	GIVEN	nem áll töltőhelyen
	WHEN	benzint tölt
	THEN	hibajelzés
pénztárhoz megy	GIVEN	létezik a benzinkút, de nem létezik pénztár
	WHEN	bemegy a pénztárba
	THEN	hibajelzés
fizet	GIVEN	nem áll töltőhelynél
	WHEN	fizet
	THEN	hibajelzés
távozik	GIVEN	nem áll töltőhelynél
	WHEN	távozik
	THEN	hibajelzés

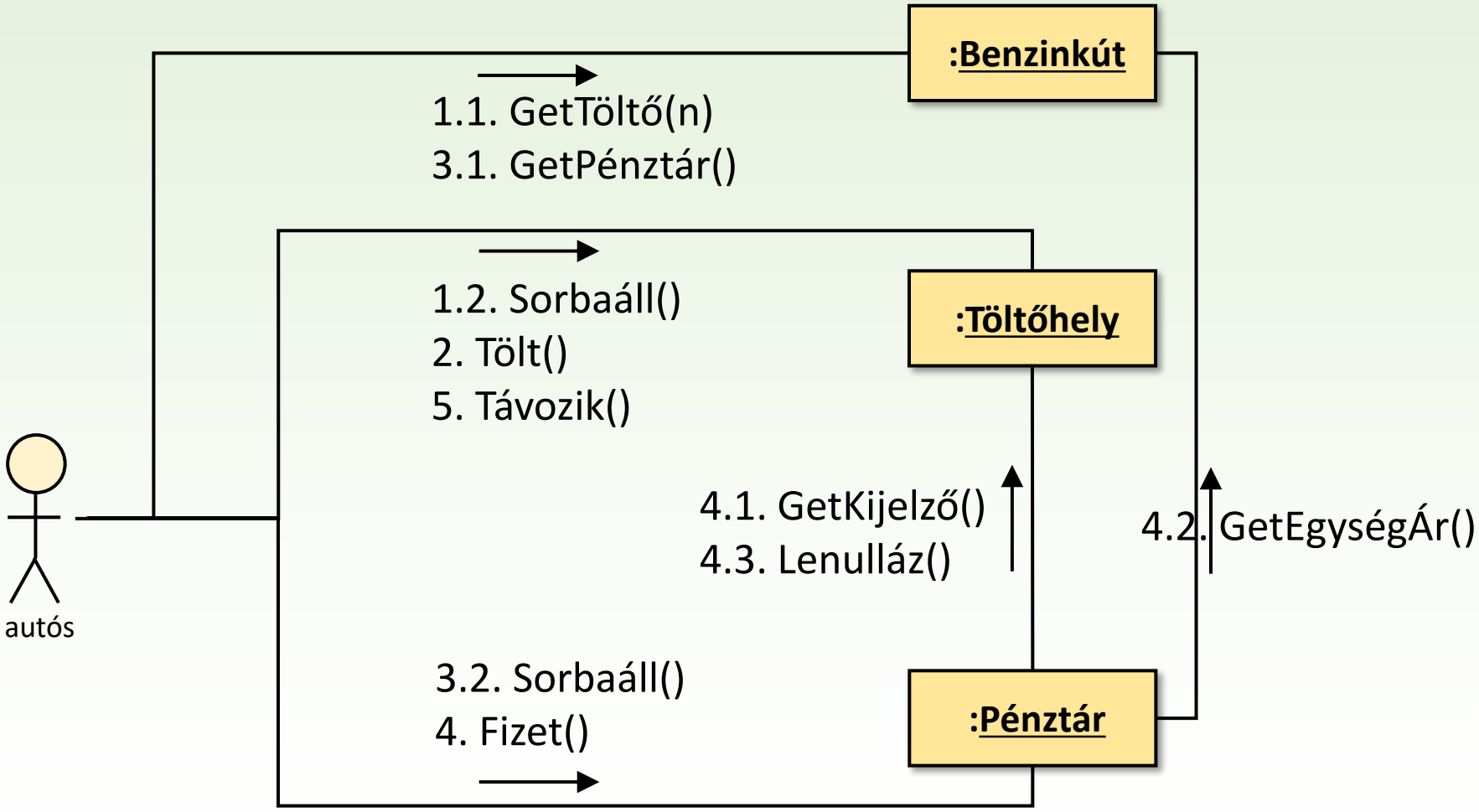
Elemzés

- Objektumok és tevékenységeik:
  - autósok (tankolnak: behajt, tölt, pénztárhoz megy, fizet, távozik)
  - benzinkút (kezeli a benzin egységárát)
  - töltőhelyek (amely mellé beáll az autós, ahol várakozik, majd benzint tölt, végül fizetés után kiáll)
  - pénztár több kasszával (ahol az autós sorba áll, majd fizet)
- Objektumok közötti kapcsolatok:
  - a benzinkút részei a töltőhelyek és a pénztár
  - egy autós ideiglenesen kapcsolatba kerül egy töltőhellyel és a pénztárral.

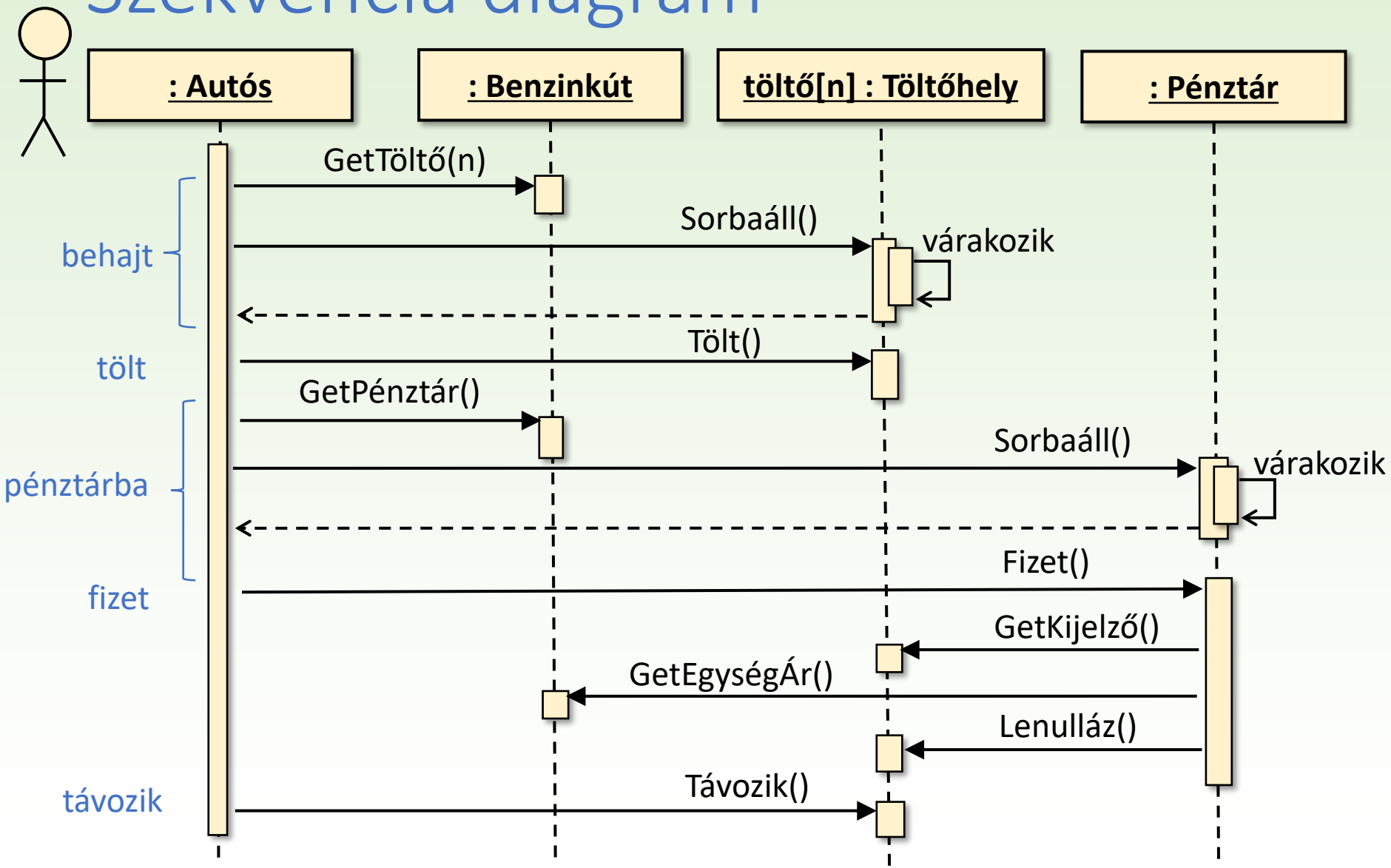
Objektum diagram



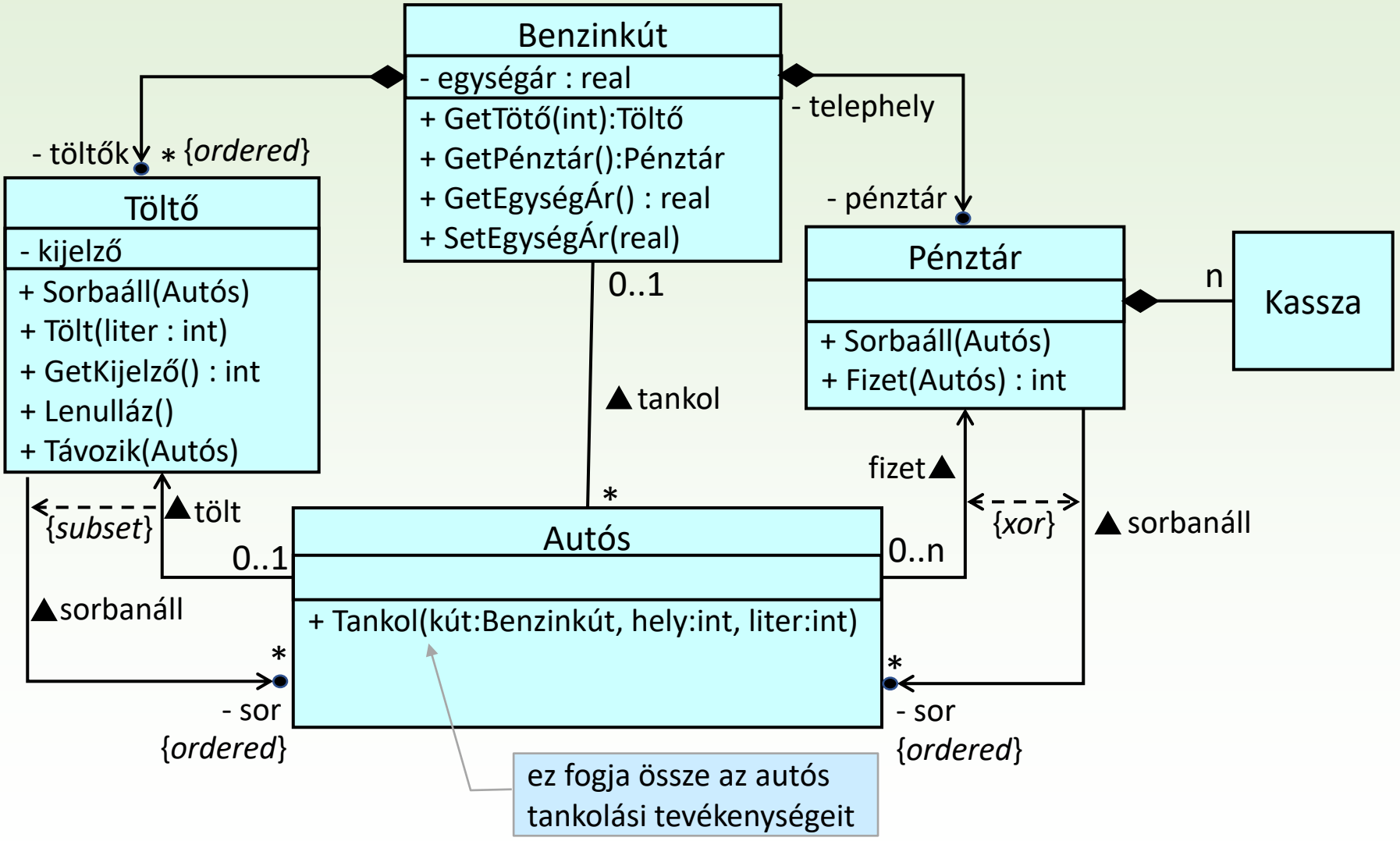
Kommunikációs diagram



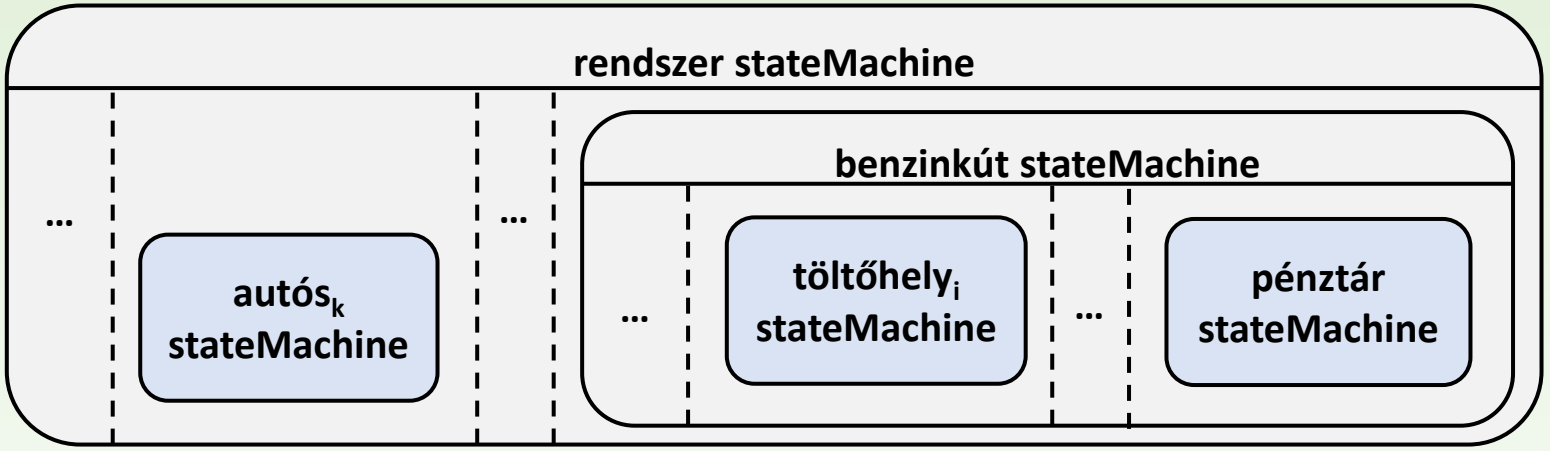
Szekvencia diagram



Osztály diagram



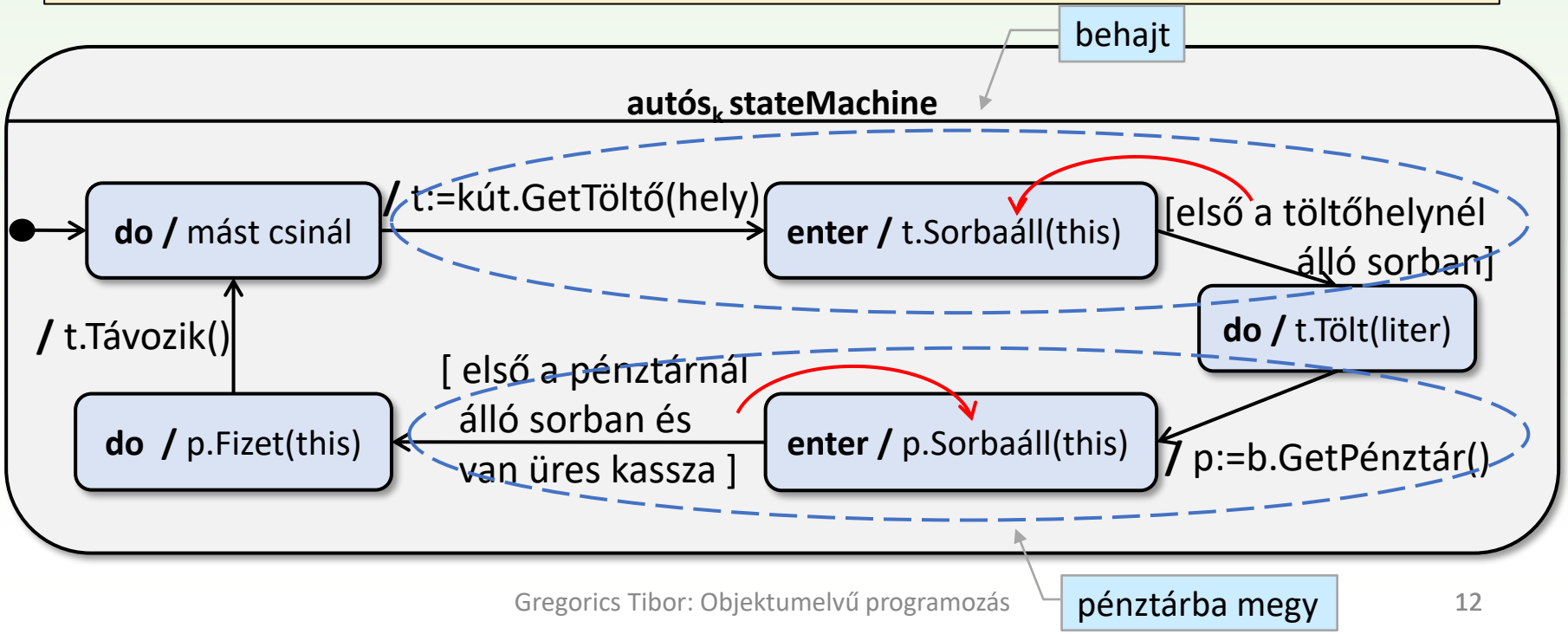
Rendszer állapotgépe



- A rendszer állapotát az autósok és a benzinkút állapota határozza meg. A benzinkút állapota a töltőhelyek és a pénztár állapotától függ.
- Az autósok „önerőből” végeznek tevékenységet, így állapotgépeik külön szálakon futnak majd.
- A benzinkút állapotgépe más objektumok által küldött szinkron üzenetek hatására működik. Nem igényel külön szálat.

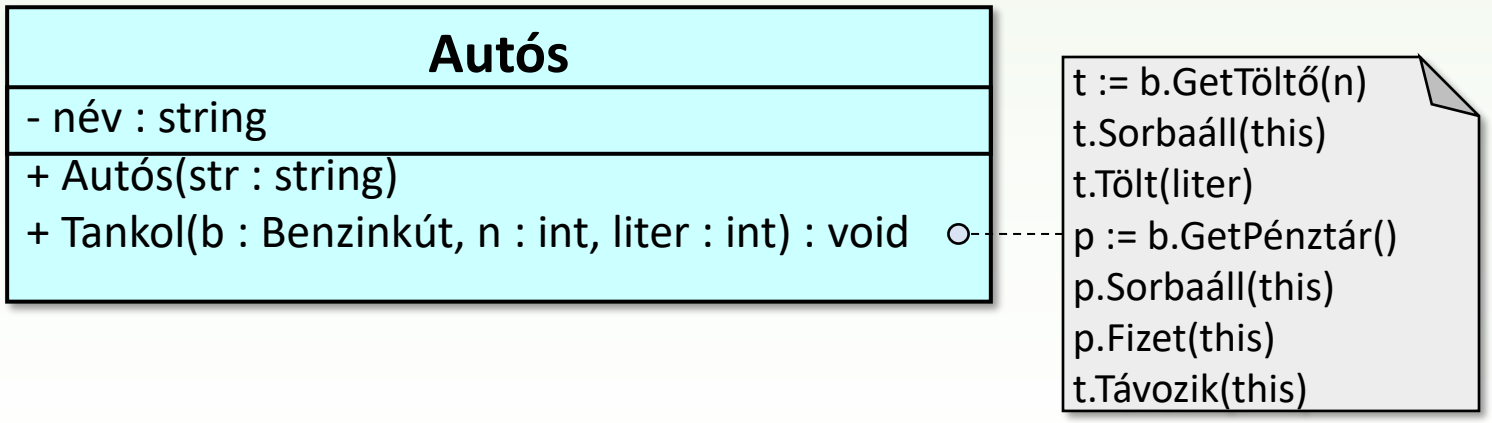
Autós objektum állapotgépe

- Az állapotok a huzamosabb ideig tartó tevékenységeket jelölik:
  - mást csinál, egy töltőhelynél sorba áll, üzemanyagot vesz fel, a pénztárban sorba áll, fizet
- Az átmenetek a rövid, pillanatnyi tevékenységeket jelölik, amelyeket nem üzenetek váltanak ki, hanem a megelőző tevékenység befejeződése, illetve őrfeltételének teljeülése.



Autós osztálya

- Az autósok saját szálukon hívják tankol() metódusukat, amely elindítja az állapotgépüket.
- A „mást csinál” tevékenységet nem implementáljuk, így az állapotgép sem alkot majd egy kört: az állapotgép ciklikusságát tankol() metódus ismételt hívásaival lehetne szimulálni.
- Eseménysor kezelésére nincs szükség



Autós osztály megvalósítása

```
class Car {
public:
	class NoRefuelException : public std::exception {};

	Car(const std::string &str) : _name(str) {}
	...
	void refuel(PetrolStation* station, unsigned int number, int liter) {
		_fuel = new std::thread(activity, this, station, number, liter);
	}
	~Car() { _fuel.join(); }
private:
	std::string _name;
	std::thread _fuel;

	void activity(PetrolStation* station, unsigned int number, int liter)
	{
		if ( nullptr==station || nullptr==station->getCash() || number<0
		    || number >=station->pumpsCount() ) throw new NoRefuelException();
		station->getPump(number)-> joinQueue(this);     // joins the pump
		station->getPump(number)->fill(this, liter);    // refuels petrol
		station->getCash()->joinQueue(this);            // goes to cash
		int sum = station->getCash()->pay(this);        // pays
		station->getPump(number)->leave(this);          // leaves the petrolstation
	}
};
```

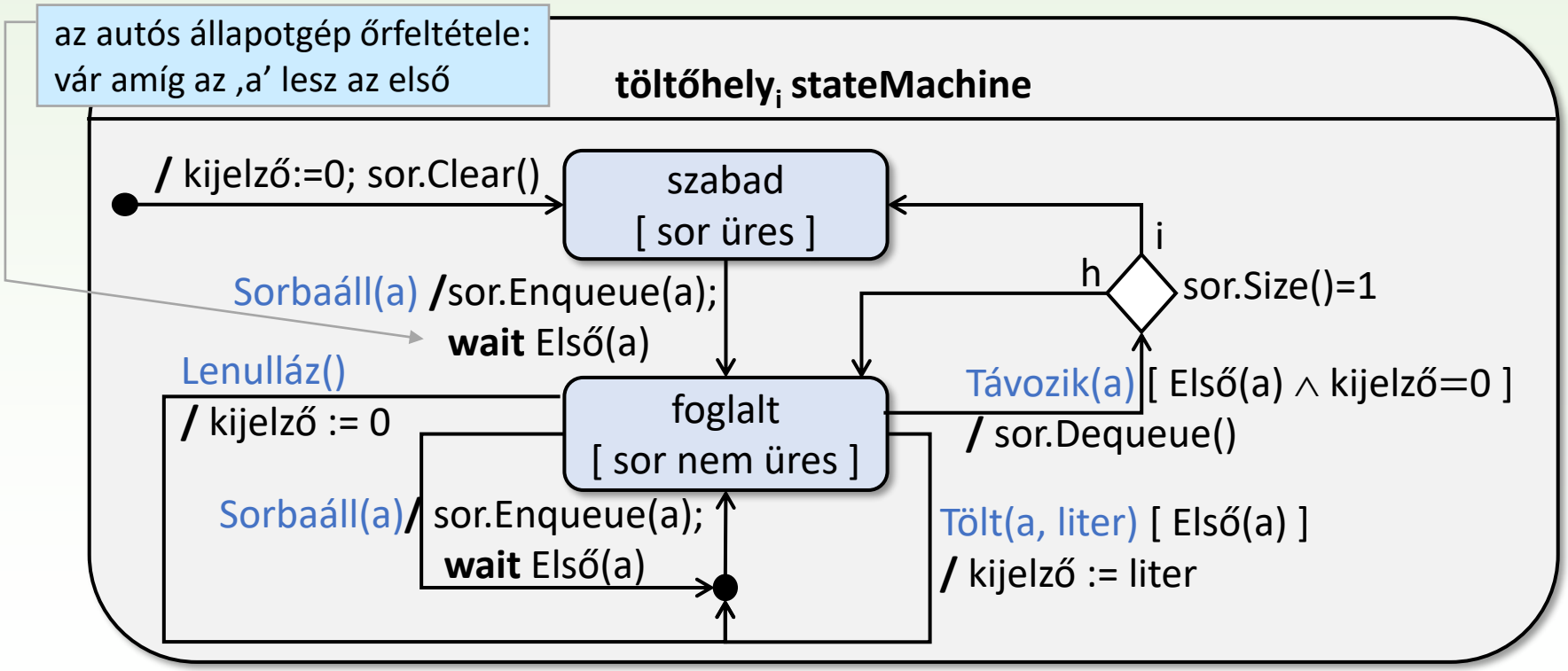
külön szálon indul el
#include <thread>

megvárja, míg a külön indított szál befejeződik

car.h

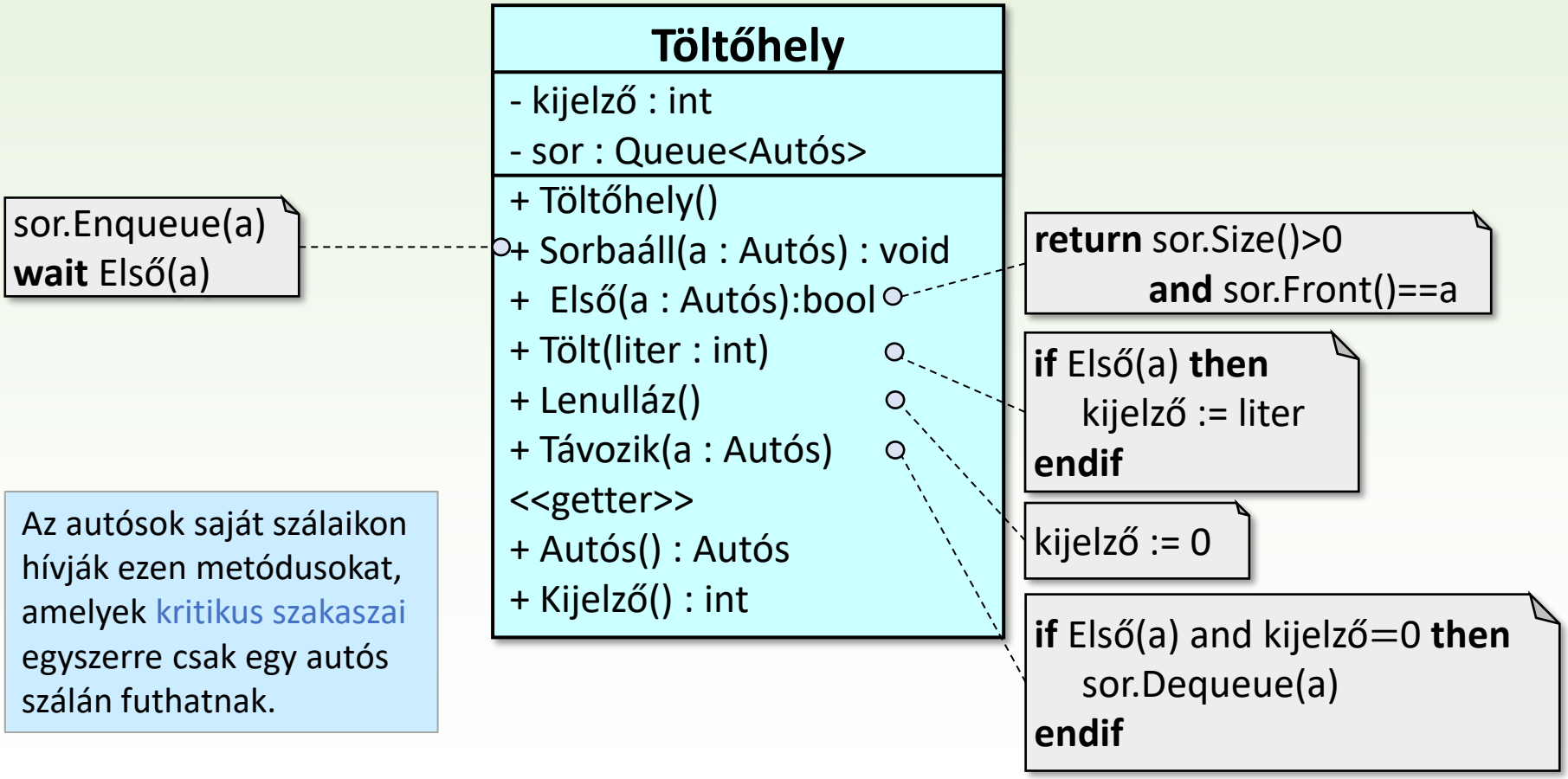
Töltőhely objektum állapotgépe

- Egy töltőhely lehet szabad vagy foglalt.
- Az állapot-átmeneteket a töltőhely szinkron módon metódus hívásai valósítják meg: a Sorbaáll() és Távozik() a töltőhelynél álló sorra van hatással, a Tölt(), Lenulláz() a töltőhely kijelzőjét módosítja.



Töltőhely osztály

- A töltőhely tulajdonságai közé tartozik a kijelzőn kívül az a sor, amelybe a töltőhelynél várakozó autósok kerülnek. A sor legelején álló autós az, aki tankolhat, illetve fizet.



Töltőhely osztály (Pump)

```
class Pump {
public:
	Pump() : _quantity(0) { }

	void joinQueue(Car * car);
	void fill(Car * car, int l);
	void leave(Car * car);

	Car* getCar() const { return _queue.front(); }
	int   getQuantity()  const {return _quantity; }
	void resetQuantity()        {_quantity = 0; }
	bool isFirst(Car *car)
	{
		return _queue.size()>0 && _queue.front()==car;
	}
private:
	std::atomic_int _quantity;
	std::queue<Car*> _queue;

	std::mutex _mu;
	std::condition_variable _cond;

};
```

szálbiztos változó #include <atomic>

A különböző szálakon futó autósok tevékenységének szinkronizálásához
#include <mutex>
#include <condition>

pump.h

Töltőhely metódusai

```
void Pump::joinQueue(Car* car)
{
      std::unique_lock<std::mutex> lock(_mu);
      _queue.push(car);    // joins the queue
      while ( !isFirst(car) ) _cond.wait(lock);
}
void Pump::fill(Car * car, int l)
{
      std::unique_lock<std::mutex> lock(_mu);
      if ( !isFirst(car) ) return;
      _mu.unlock();
      _quantity = l;   // fueling
      std::mutex _mu1;
      std::condition_variable _cond1;
      std::unique_lock<std::mutex> lock1(_mu1);
      _cond1.wait_for(lock1, milliseconds(l*100));
}
void Pump::leave(Car * car)
{
      std::unique_lock<std::mutex> lock(_mu);
      if( isFisrt(car) && 0==_quantity ) _queue.pop();   // quits the queue
      _cond.notify_all();
}
```

kritikus szakasz eleje

aktuális szál végrehajtásának felfüggesztése, hogy más szál is kritikus szakaszba tudjon lépni

kritikus szakasz vége

kritikus szakasz eleje

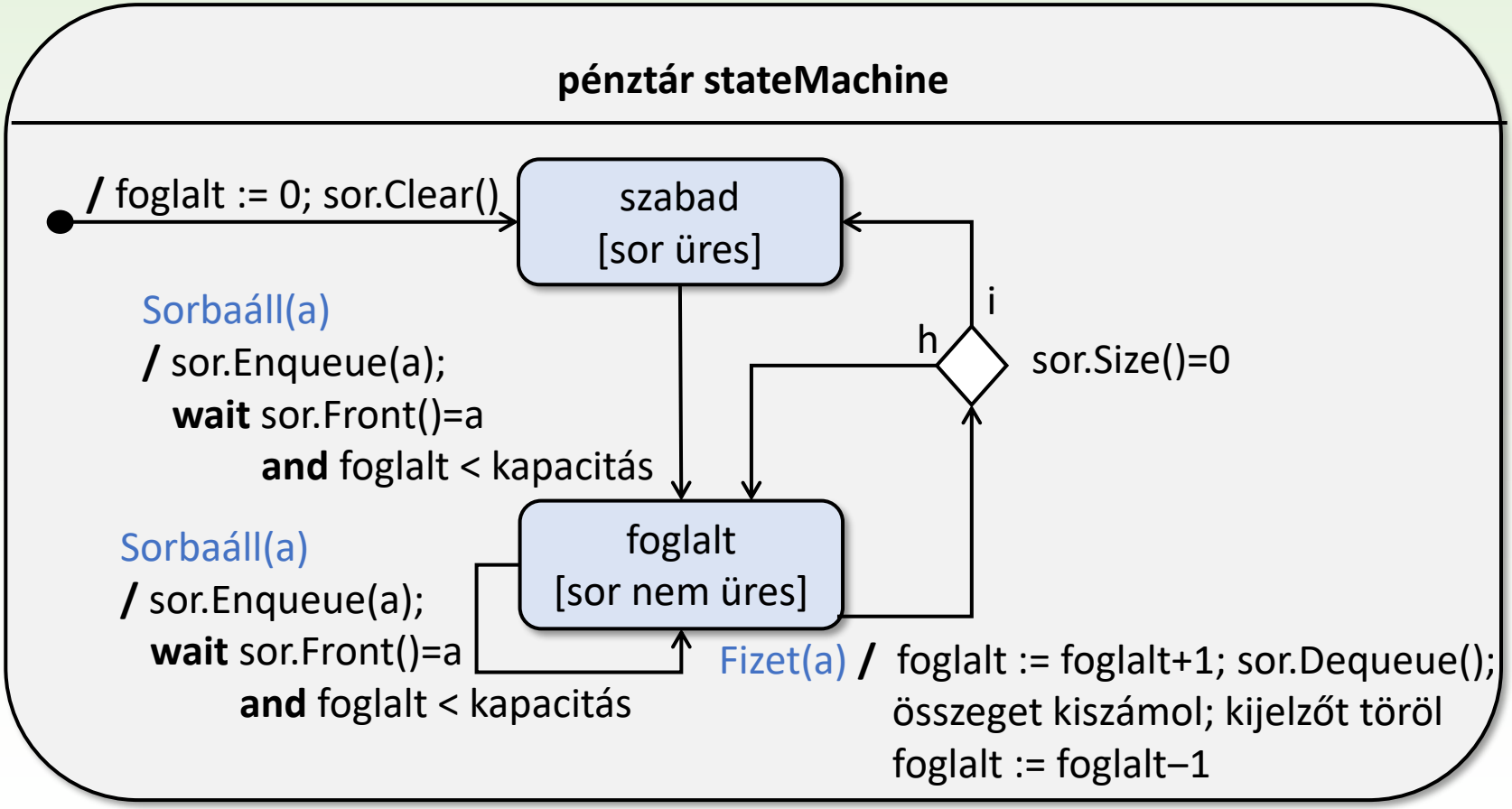
kritikus szakasz vége

elindítja a cond-nál várakozó autósok szálait, de a kritikus szakaszban levőket egyesével.

pump.cpp

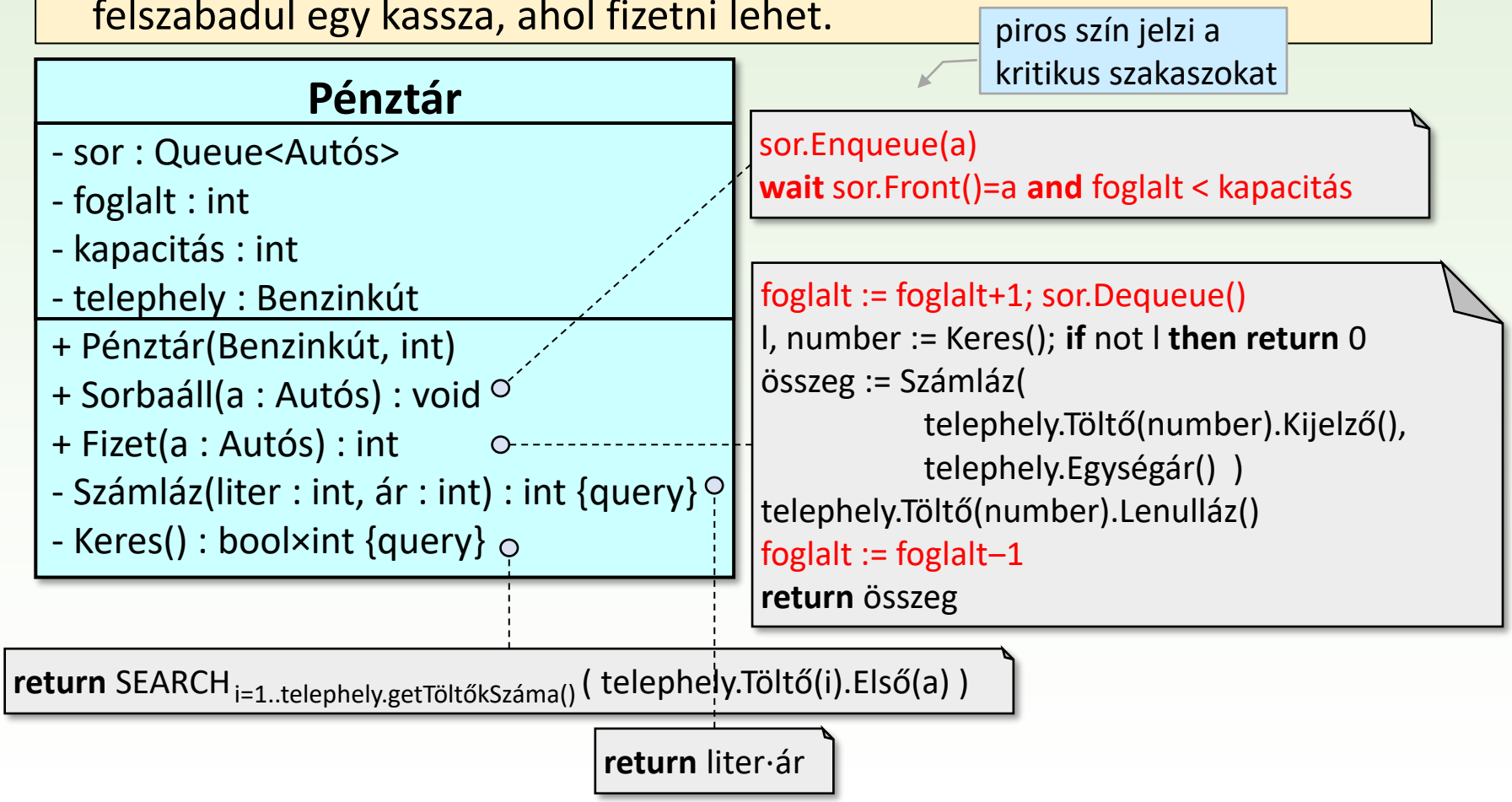
Pénztár objektum állapotgépe

- A pénztár lehet szabad vagy foglalt. Az állapot-átmeneteket a pénztár szinkron hívott metódusai valósítják meg.



Pénztár osztály

- Ismert a kasszák száma (kapacitás), a fogalt kasszák száma (foglalt), és az autósoknak a pénztárhoz álló sora, amelyből az első akkor lép ki, amikor felszabadul egy kassza, ahol fizetni lehet.



Pénztár osztály

```
class Cash
{
public:
      Cash(PetrolStation * ps, int m): _ station(ps), _capacity(m) {}
      void joinQueue(Car *car);
      int   pay(Car *car);

private:
      PetrolStation *_station;
      std::atomic_int _capacity;
      std::atomic_int _engaged;
      std::queue<Car*> _queue;

      bool search(Car *car, unsigned int &i) const
      {
         for(i = 0; i<_station->pumpsCount(); ++i) {
            if ( _station->getPump(i)->getCar() == car ) return true;
         }
         return false;
      }
      int   invoice(int liter, int price) const { return liter*price; }

      std::mutex _mu;
      std::condition_variable _cond;
};
```

megkeresi azt a töltőhelyet, ahol az autós tankolt

cash.h

Pénztár metódusai

```
void Cash::joinQueue(Car* car)
{
	std::unique_lock<std::mutex> lock(_mu);
	_queue.push(car);   // joins the queue
	while ( ! (_queue.size()>0 && _queue.front()==car) || _engaged==_capacity )
		_cond.wait(lock);
}

int Cash::pay(Car* car)
{
	std::unique_lock<std::mutex> lock(_mu);
	++_engaged;		// steps to a cash desk
	_queue.pop();		// leaves the queue
	_mu.unlock();

	unsigned int number;
	if (!search(car, number)) return 0;
	int sum = invoice(_station->getPump(number)->getQuantity(), _station->getUnit());
	_station->getPump(number)->resetQuantity();  // resets the pump
	...			//elapsed time of paying
	--_engaged;		// leaves the cash desk
	_cond.notify_all();
	return sum;
}
```

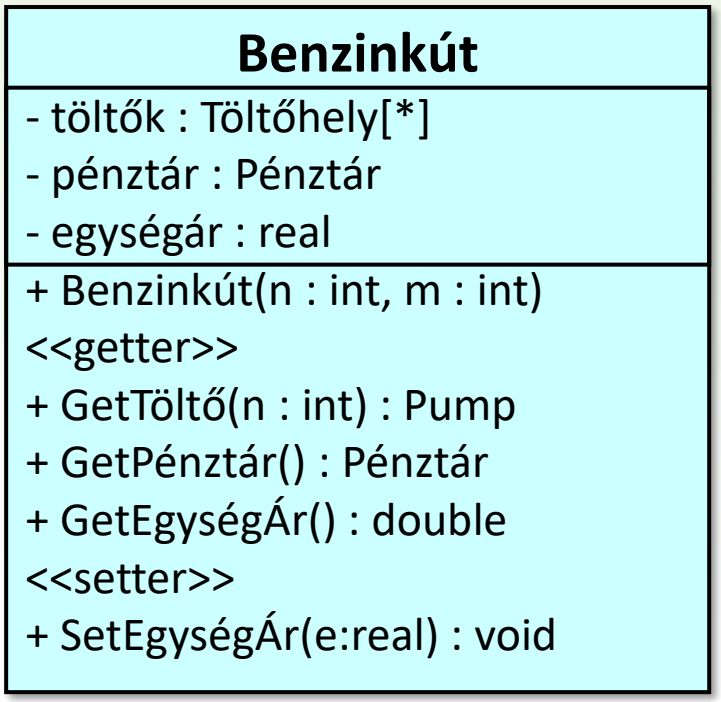
ha az autós nem a legelső a pénztár sorában vagy nincs szabad kassza, akkor várakozik

tovább engedi az összes várakozó autós szálát, de csak egyesével.

cash.cpp

Benzinkút osztálya

- A benzinkút getter-eket szolgáltat a rendszer komponenseinek (töltőhelyek, pénztár) eléréséhez, valamint getter-t és setter-t az egységár adattaghoz.



Benzinkút osztálya

```
class PetrolStation {
public:
	PetrolStation(int n, int m) {
		for(int i = 0; i<n; ++i) _pumps.push_back( new Pump() );
		 _cash = new Cash(this, m);
	}
	~PetrolStation() { for( Pump *p : _pumps ) delete p;  delete _cash; }

	Pump* getPump(unsigned int number) const { return _pumps[number]; }
	unsigned int pumpsCount() const { return _pumps.size(); }
	Cash* getCash() const { return _cash; }

	int getUnit() const { return _unit; }
	void setUnit(int u) { _unit = u; }

private:
	std::vector<Pump*> _pumps;
	Cash *_cash;
	int _unit;
};
```

station.h

Fejlesztő eszközök

Gregorics Tibor

gt@inf.elte.hu

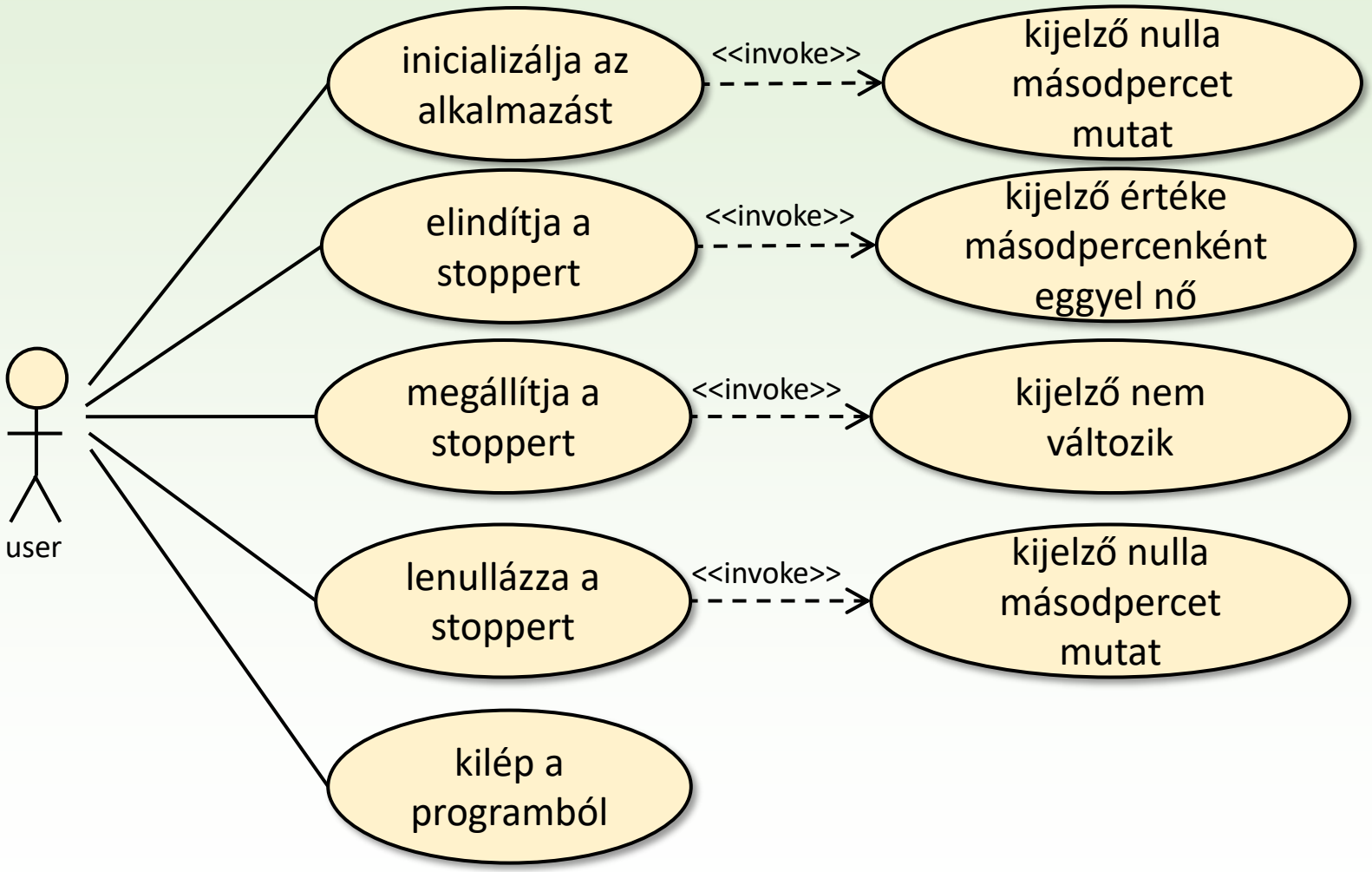
http://people.inf.elte.hu/gt/oep

Feladat: Stopper

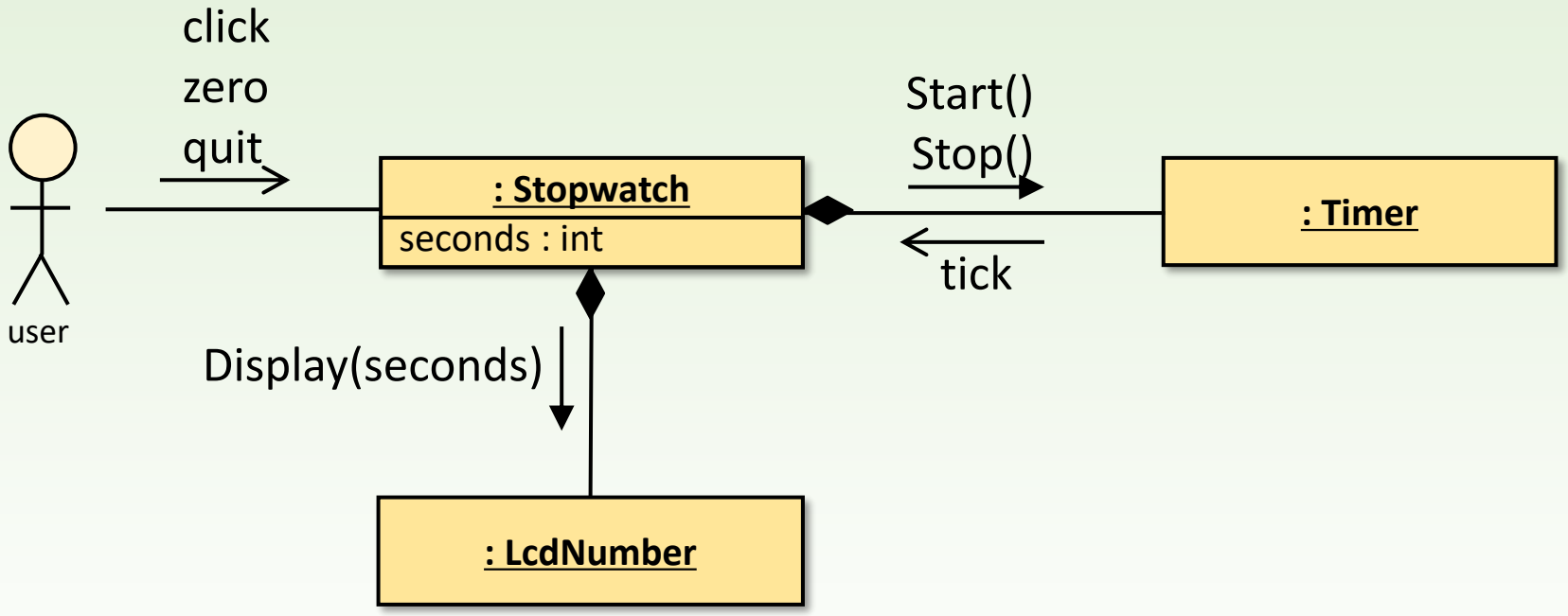
Készítsünk egy stoppert, amely másodpercenként méri a múló időt.

- A mérés egy jelzés hatására induljon el, majd egy ugyanilyen jelzés hatására álljon le; majd újabb jelzés hatására folytatódjon, és így tovább.
- Legyen lehetőség a kijelzett idő lenullázására, amely a mérést is megállítja.
- Külön jelzés hatására az alkalmazás álljon le.

Használati eset diagram

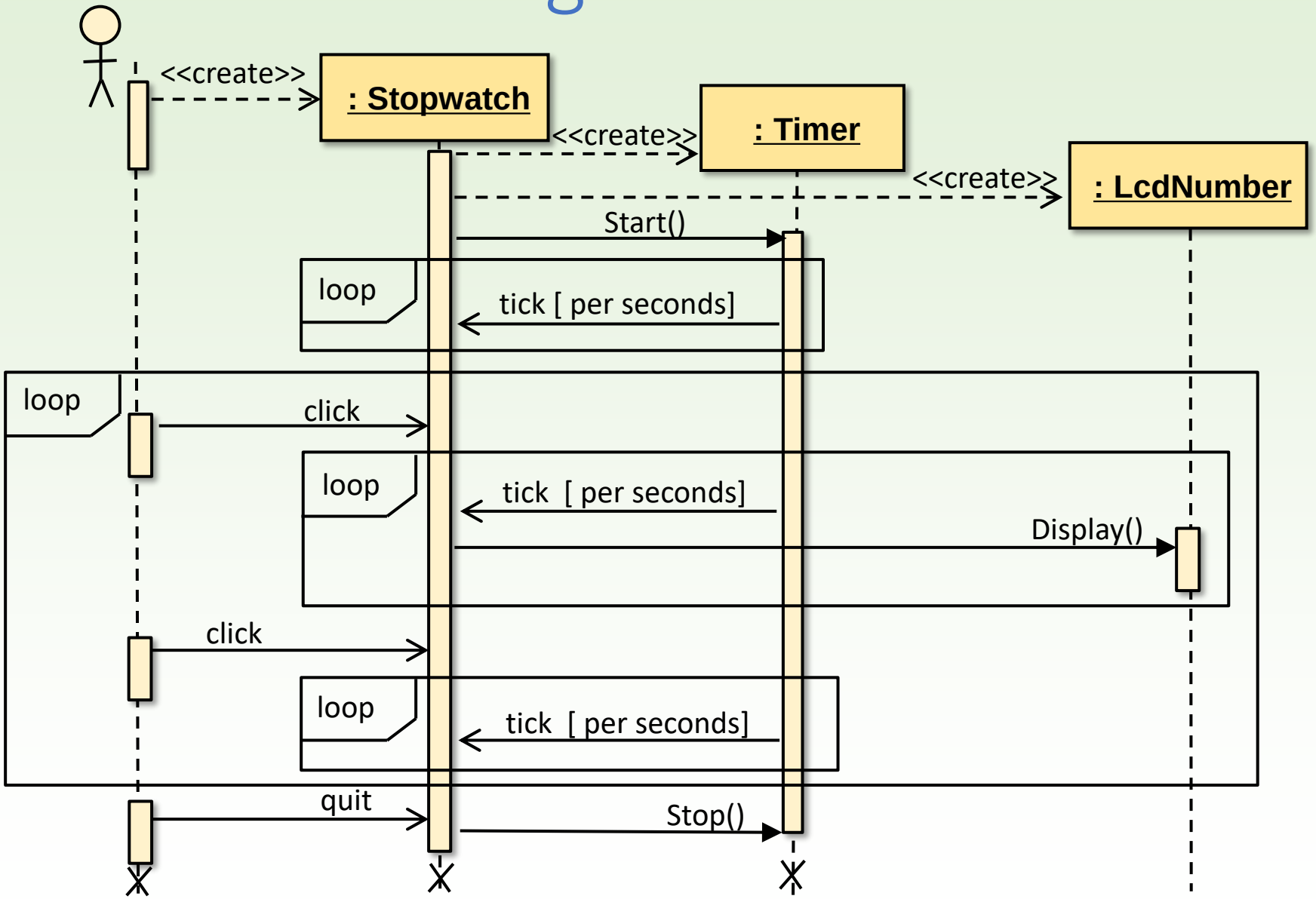


Kommunikációs és objektum diagram

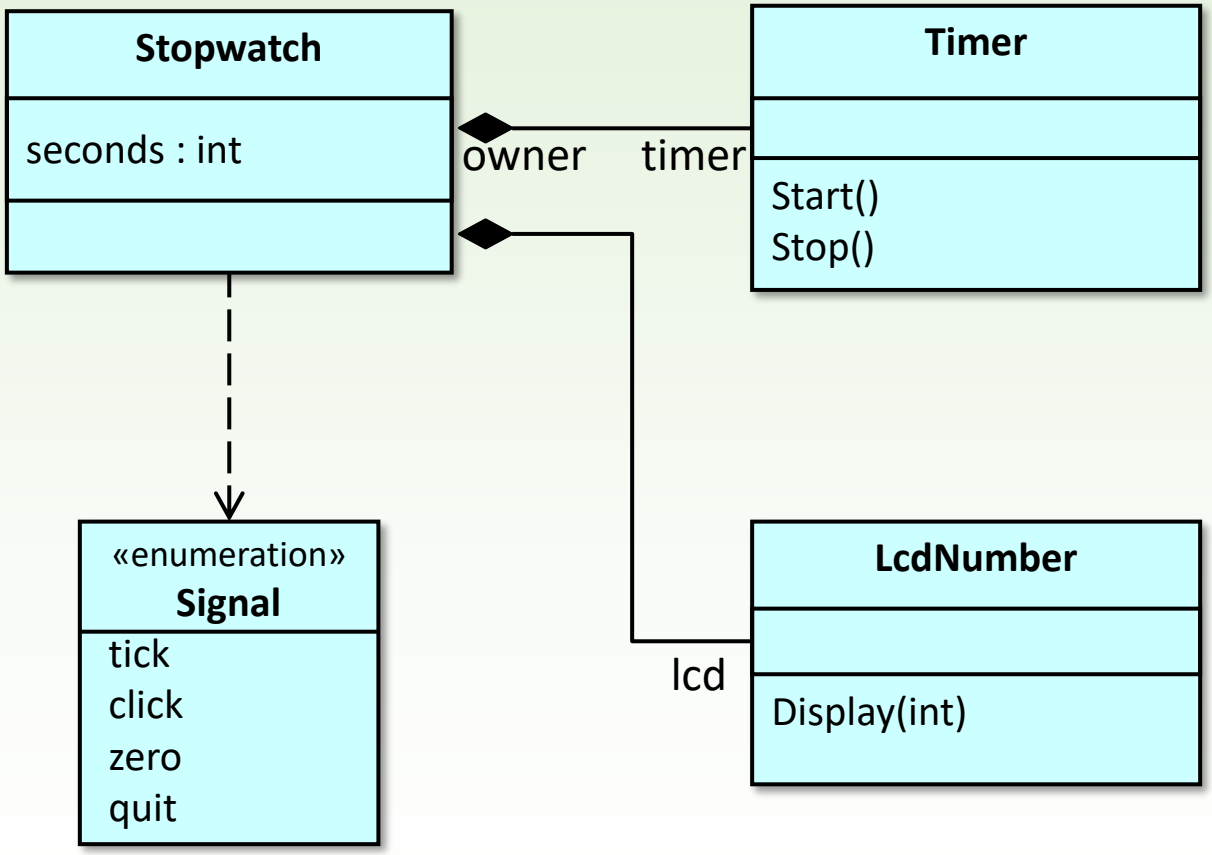


- a felhasználó és az időzítő szignálokat küld a stoppernek
- a stopper a neki küldött szignálokat aszinkron módon dolgozza fel.

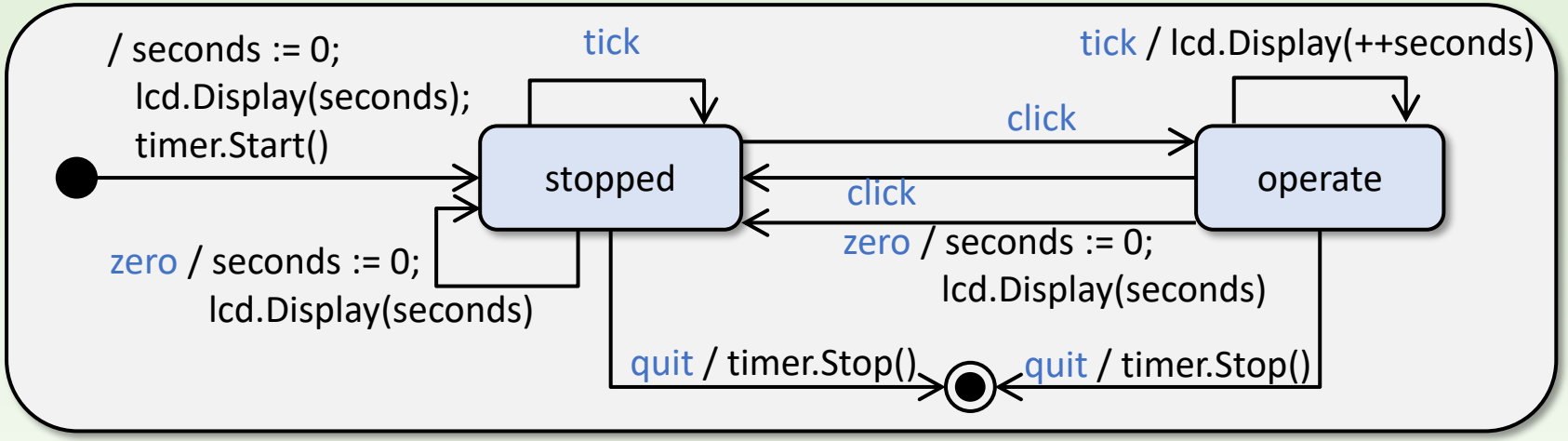
Szekvencia diagram



Osztálydiagram

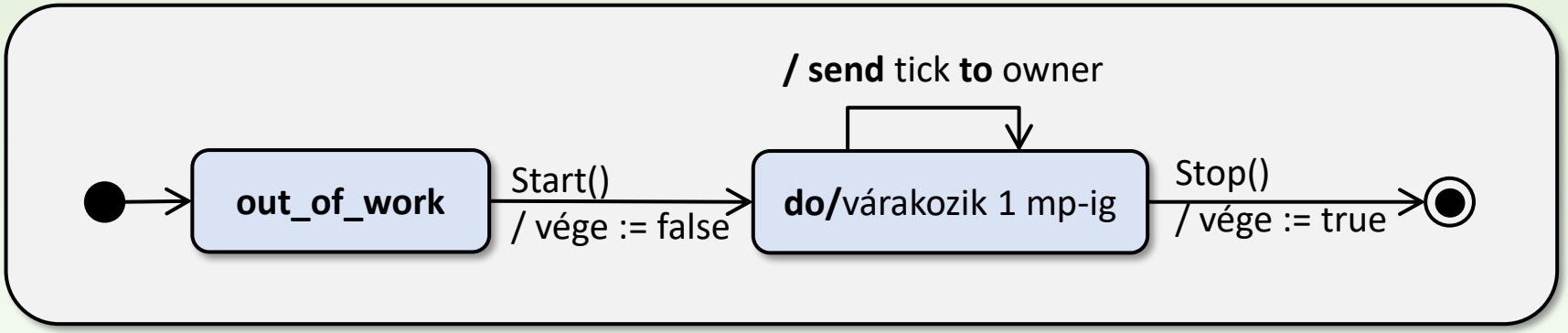


Stopwatch állapotgépe



állapot / esemény	stopped	operate
click	operate	stopped
tick	stopped	/ lcd.Display(++seconds) operate
zero	/ second:=0; lcd.Display(seconds) stopped	/ second:=0; lcd.Display(seconds) stopped
quit	/ timer.Stop() final	/ timer.Stop() final

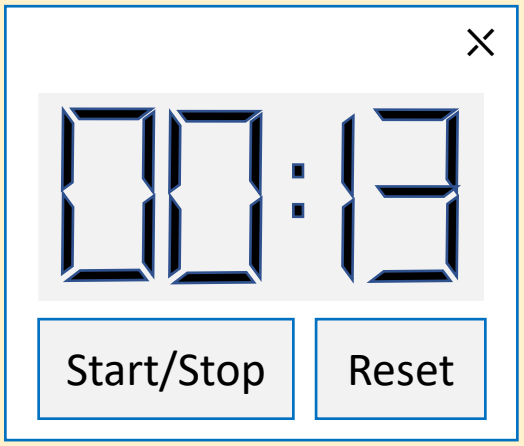
Timer állapotgépe




```
vége := false
while not vége loop
    wait 1000
    send tick to owner
endloop
```

Megvalósítás

- Implementálhatnánk a megoldást az eddig látott módon: a szálkezelést, a szálbiztos eseménysorral támogatott eseménykezelést, az őrfeltételek várakozó utasítását magunk építjük be a kódba, pedig mennyivel gyorsabb és egyszerűbb lenne a fejlesztés, ha mindez „automatikusan” kerülne bele alkalmazásba.



- Eddig konzolos alkalmazásokat készítettünk, de szebb lenne grafikus megjelenést használni.
- Olyan fejlesztő környezetet keresünk, amely
  - hozzáad az alkalmazáshoz egy aszinkron eseménykezelő mechanizmust
  - lehetőséget ad grafikus megjelenéssel rendelkező objektumok létrehozására úgy, hogy azok egyedi tulajdonságait meg lehessen megadni
  - vizuális tervezőt biztosít a grafikus megjelenítés kialakításához
  - speciális objektumok (pl. időzítő) létrehozására ad kényelmes lehetőséget

Stopwatch .net alatt fejlesztve



Stopwatch : Form egy ablakszerű vezérlő objektum osztálya, amely más vezérlőket (időzítő, kijelző, nyomógomb) tartalmaz. Az ablak bezárása váltja ki a quit szignált.

TextBox osztály példánya a kijelző, amelyhez egy display metódust kell írni.

Button típusú objektumok, amelyek a click illetve a zero szignált váltják ki.

a háttérben lesz egy **Timer** típusú objektum, amely tick szignált vált ki

Application osztály statikus metódusai gondoskodnak arról, hogy az események a megfelelő vezérlőkhöz jussanak el, hogy ott szignálokat váltsanak ki.

```
static class Program
{
    [STAThread]
    static void Main(){
        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault(false);
        Application.Run(new Stopwatch());
    }
}
```

program.cs

Stopwatch osztály, mint .net Form

```
public class Stopwatch : Form
{
    enum State { stopped, operate };
    State currentState;
    DateTime seconds = new DateTime(0);

    private System.Windows.Forms.Timer   timer;
    private System.Windows.Forms.Button  clickButton;
    private System.Windows.Forms.Button  resetButton;
    private System.Windows.Forms.TextBox lcd;

    public Stopwatch()
    {
       InitializeComponent();
       currentState = State.stopped;
       display();
       timer.Start();
    }

    private void display() { … }

    private void timer_Tick(object sender, EventArgs e) { … }
    private void clickButton_Click(object sender, EventArgs e) { … }
    private void resetButton_Click(object sender, EventArgs e) { … }
    private void MainForm_FormClosed(object sender, FormClosedEventArgs e)
    { timer.Stop(); }
}
```

szignálok eseménykezelői (lásd állapot-átmenet tábla megfelelő sorait)

Stopwatch.cs

Visual Studio-val tervezett kód

```
public InitializeComponent()
{
   this.components  = new System.ComponentModel.Container();
   this.clickButton = new System.Windows.Forms.Button();
   this.resetButton = new System.Windows.Forms.Button();
   this.lcd         = new System.Windows.Forms.TextBox();
   this.timer       = new System.Windows.Forms.Timer(this.components);
   ...
   this.Text = "Stopwatch";
   this.clickButton.Text = "Start/Stop";
   this.resetButton.Text = "Reset";
   this.lcd.Text = "00:00";
   this.timer.Interval = 1000;
   ...
   this.Controls.Add(this.lcd);
   this.Controls.Add(this.clickButton);
   ...
   this.timer.Tick   += new System.EventHandler(this.timer_Tick);
   this.zeroButton.Click += new System.EventHandler(this.resetButton_Click);
   this.clickButton.Click += new System.EventHandler(this.clickButton_Click);
   this.FormClosed   += new System.Windows.Forms.
                        FormClosedEventHandler(this.MainForm_FormClosed);
}
```

a kód ezen része generálható az automatikusan vizuális tervezéssel

szignálok és kezelőik egymáshoz rendelése

Stopwatch.cs

Stopwatch Java-ban



Stopwatch extends JFrame egy ablakszerű vezérlő objektum osztálya, amely más vezérlőket (időzítő, kijelző, nyomógomb) tartalmaz. Az ablak bezárása váltja ki a quit szignált.

LCDNumber osztály példánya a kijelző, amely display metódussal rendelkezik.

JButton típusú objektumok, amelyek a click illetve a zero szignált váltják ki.

a háttérben lesz egy **Timer** típusú objektum, amely tick szignált vált ki

```
public class Stopwatch extends JFrame
{
        ...
    public static void main(String[] args) {
        new Stopwatch();
    }
}
```

Stopwatch.java

Stopwatch osztály, mint JFrame

```
public class Stopwatch extends JFrame
{
        enum State { operate, stopped }
        private State currentState;
        private int seconds = 0;

        private final static int SECOND = 1000 /* milliseconds */;
        private Timer timer = new Timer(SECOND, null);
        private LcdNumber lcd = new LcdNumber("00:00");
        private JButton clickButton = new JButton("Start/Stop");
        private JButton resetButton = new JButton(„Reset");
        private JPanel buttonPanel = new JPanel();

        public Stopwatch() { … }

        void click() { … }
        void reset() { … }
        void tick() { … }
        protected void finalize() throws Throwable { … timer.stop() … }

        public static void main(String[] args) {
                new Stopwatch();
        }
}
```

szignálok eseménykezelői (lásd állapot-átmenet tábla megfelelő sorait)

Stopwatch.java

Java-s Stopwatch konstruktora

```
public Stopwatch() {
        super("Stopwatch");
        setBounds(250, 250, 300, 200);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        buttonPanel.setBackground(Color.WHITE);
        buttonPanel.add(clickButton);
        buttonPanel.add(resetButton);
        add(lcd);
        add(buttonPanel, "South");

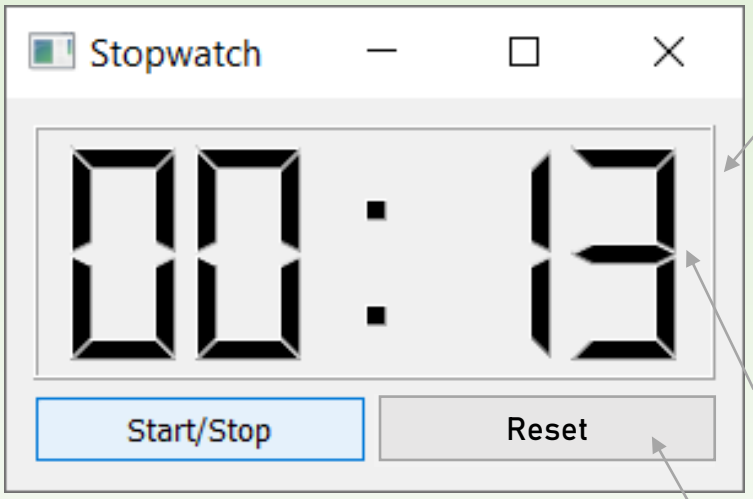
        clickButton.addActionListener(new ActionListener() {
                @Override public void actionPerformed(ActionEvent e){ click(); }
        });
        zeroButton.addActionListener(new ActionListener() {
                @Override public void actionPerformed(ActionEvent e){ reset(); }
        });
        timer.addActionListener(new ActionListener() {
                @Override public void actionPerformed(ActionEvent e){ tick(); }
        });
        currentState = State.stopped;
        lcd.display(seconds);
        timer.start();
        setVisible(true);
}
```

quit

események és kezelésük egymáshoz rendelése

Stopwatch.java

Stopwatch Qt-val fejlesztve



Stopwatch : public Qwidget egy ablakszerű vezérlő objektum osztálya, amely más vezérlőket (időzítő, kijelző, nyomógomb) tartalmaz. Az ablak bezárása váltja ki a quit szignált.

QLCDNumber osztály példánya a kijelző, amely display metódussal rendelkezik.

QPushButton típusú objektumok, amelyek a click illetve a zero szignált váltják ki.

a háttérben lesz egy **QTimer** típusú objektum, amely tick szignált vált ki

Qapplication típusú objektum többek között gondoskodik arról, hogy az események a megfelelő vezérlőkhöz jussanak el, hogy ott szignálokat váltsanak ki.

```
#include <QApplication>
#include "stopwatch.h"
int main(int argc, char *argv[])
{
        QApplication app(argc,argv);
        Stopwatch *stopwatch = new Stopwatch;
        stopwatch ->show();
        return app.exec();
}
```

main.cpp

Stopwatch osztály, mint QWidget

```
#include <QWidget>
enum State {stopped, operate};

class Stopwatch: public Qwidget
{
        Q_OBJECT
    private:
        QTimer          *_timer;
        QLCDNumber  *_lcd;
        QPushButton   *_clickButton;
        QPushButton   *_resetButton;
        State _currentState;
        int   _seconds;
        QString Stopwatch::format(int n) const;
    public:
        Stopwatch(QWidget *parent=0);
    private slots:
        void oneSecondPass();
        void clickButtonPressed();
        void resetButtonPressed();
    protected:
        void closeEvent(QCloseEvent * event) { _timer->stop(); }
};
```

tick, click, zero szignálok eseménykezelői (lásd állapot-átmenet tábla megfelelő sorait)

quit szignál eseménykezelője

stopwatch.h

Qt-s Stopwatch konstruktora

```
Stopwatch::Stopwatch(QWidget *parent) : Qwidget(parent)
{

        setWindowTitle(tr("Stopwatch"));
        resize(150, 60);

        _timer  = new QTimer;
        _lcd    = new QLCDNumber;
        _clickButton = new QPushButton("Start/Stop");
        _resetButton = new QPushButton("Reset");

        …

        connect(_timer,  SIGNAL(timeout()), this, SLOT(oneSecondPass()));
        connect(_clickButton, SIGNAL(clicked()), this, SLOT(clickButtonPressed()));
        connect(_resetButton, SIGNAL(clicked()), this, SLOT(resetButtonPressed()));


        _currentState = stopped;
        _seconds = 0;
        _lcd->display(_seconds);
        _timer->start(1000);
}
```

itt kerül sor a grafikus vezérlőknek
az ablakban való elrendezésére
és egyéb tulajdonságainak megadására.
Ez automatikusan is generálható,
ha vizuális tervezőt (QtDesigner) használunk

szignálok és kezelőik egymáshoz rendelése:

stopwatch.cpp